Deep Learning: Autoencoders, Variational Autoencoders and GANs

Johannes Lutzeyer

Data Science and Mining Team (DASCIM), LIX
École Polytechnique

October 2026

These slides are based on original slides provided by **Prof. Michalis Vazirgiannis**.

GPT-6 Astra Medium was used in the process of modifying the original slides of Prof. Vazirgiannis.

Outline

1. Autoencoders (AEs)
2. Variational Autoencoders (VAEs)
3. Generative Adversarial Networks (GANs)

Notation Used Throughout the Lecture

$x_i \in \mathbb{R}^d$	Observation i; N observations in the training set.
$z \in \mathbb{R}^m$	Latent code; m is the latent dimension.
f_ϕ, $q_\phi(z \mid x)$	Deterministic AE encoder; probabilistic VAE encoder.
g_θ, $p_\theta(x \mid z)$	Deterministic AE decoder; probabilistic VAE decoder.
G_θ, D_ψ	GAN generator; GAN discriminator.
$p(z)$	Latent prior, usually $\mathcal{N}(0, I_m)$.
ℓ, J	Per-observation loss; training objective to minimise.
$\mathcal{L}_{\theta,\phi}$	VAE lower bound to maximise.

Can We Learn Something Useful by Copying an Image?

Imagine sending a handwritten digit through a narrow channel

The sender can transmit only a short code. The receiver must reconstruct the image from that code. What information should the sender preserve?

- The **encoder** turns the image into a code; the **decoder** *reconstructs* the image from that code.
- Train both networks by comparing the reconstruction with the original image. No digit labels are needed.
- A bottleneck or regularisation makes the task more useful than unrestricted copying.

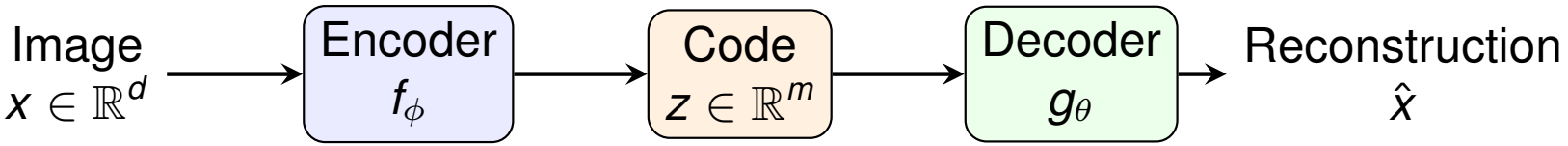
Question

If the network could simply copy every pixel, what would encourage it to learn useful features?

Background: Goodfellow et al. [GBC16]; Hinton and Salakhutdinov [HS06].

From an Image to a Code, and Back Again

A 28×28 grayscale image is a vector $x \in \mathbb{R}^{784}$. We might encode it using just 32 numbers, $z \in \mathbb{R}^{32}$.



Here d is the input dimension and m the code dimension; ϕ and θ are the networks' learned parameters.

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z).$$

For N training images, minimise their average reconstruction loss:

$$J_{\mathrm{AE}} = \frac{1}{N} \sum_{i=1}^{N} \ell(x_i, \hat{x}_i), \qquad \text{e.g.,} \quad \ell(x, \hat{x}) = \tfrac{1}{2} \|x - \hat{x}\|_2^2.$$

Undercomplete: $m < d$; **overcomplete:** $m > d$. A narrow code alone does not guarantee useful features.

Goodfellow et al. [GBC16], Chapter 14.

Applications and the Connection to PCA

- **Dimensionality reduction:** learn a compact representation of observations.
- **Feature learning:** use encoded features for classification, retrieval or clustering; benefits must be evaluated on the downstream task.
- **Denoising:** reconstruct clean observations from corrupted inputs using a denoising training objective.

When does an AE recover PCA?

For centred data, linear encoder and decoder, squared reconstruction loss and a rank-m bottleneck, a global optimum reconstructs the projection onto a leading m-dimensional principal subspace.

Nonlinear AEs can model more general structure, but need not outperform PCA on a given dataset.

Goodfellow et al. [GBC16]; Hinton and Salakhutdinov [HS06]; Vincent et al. [V10].

Sparse Autoencoders: Penalising Average Activation

Suppose we want each code unit to be active only rarely. With sigmoid activations, unit j outputs $a_j(x) = [f_\phi(x)]_j \in (0, 1)$. Its average is

$$\hat{\rho}_j = \frac{1}{N} \sum_{i=1}^{N} a_j(x_i), \qquad \rho = 0.02 \quad \text{is an example target average.}$$

Penalise departures from a chosen target $0 < \rho < 1$ (usually small):

$$R_{\text{sp}} = \sum_{j=1}^{m} \left[\rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \right].$$

$$J_{\text{sparse}} = J_{\text{AE}} + \lambda_{\text{sp}} R_{\text{sp}}, \qquad \lambda_{\text{sp}} \geq 0 \text{ controls the penalty strength.}$$

- Encourages a low *average* activation for each unit; it does not enforce exact zeros or a fixed number of active units per sample.
- This Bernoulli penalty is appropriate for activations in $(0, 1)$; an ℓ_1 penalty is another option for other code types.

Ng [Ng11]. This is a sum of Bernoulli KL divergences; it is zero when $\hat{\rho}_j = \rho$ for all j. All logarithms are natural.

Sparse Autoencoders: Backpropagating the Penalty

Let $a_j(x_i) = [f_\phi(x_i)]_j \in (0, 1)$ denote the activation of code unit j for input x_i. For the objective on the previous slide,

$$\frac{\partial R_{\text{sp}}}{\partial \hat{\rho}_j} = -\frac{\rho}{\hat{\rho}_j} + \frac{1-\rho}{1-\hat{\rho}_j}, \qquad \frac{\partial \hat{\rho}_j}{\partial a_j(x_i)} = \frac{1}{N}.$$

By the chain rule, multiply these two terms:

$$\frac{\partial R_{\text{sp}}}{\partial a_j(x_i)} = \frac{\partial R_{\text{sp}}}{\partial \hat{\rho}_j} \frac{\partial \hat{\rho}_j}{\partial a_j(x_i)} = \frac{1}{N}\left(-\frac{\rho}{\hat{\rho}_j} + \frac{1-\rho}{1-\hat{\rho}_j}\right).$$

- Add the sparsity contribution to the reconstruction gradient, then continue backpropagation through the encoder.
- If $\hat{\rho}_j > \rho$, the sparsity contribution is positive: gradient descent pushes activations down.

Autoencoders for Clustering: DEC

Can we group images using the codes we have learned?

Deep Embedded Clustering (DEC) refines the encoder f_ϕ together with K cluster centres $c_1, \ldots, c_K$:

$$z_i = f_\phi(x_i), \qquad c_j \in \mathbb{R}^m, \quad j = 1, \ldots, K.$$

1. Pretrain an autoencoder and initialise K centres with K-means on its codes.
2. Discard the decoder; refine the encoder and centres with a clustering loss.

The next question

How can we assign a code to clusters smoothly enough to learn the encoder by backpropagation?

Xie, Girshick and Farhadi [XGF16].

DEC: Soft Assignments and Auxiliary Targets

Let r_{ij} be the soft assignment of code z_i to centre c_j. A Student-t kernel, with shape parameter $\alpha > 0$, gives

$$r_{ij} = \frac{\left(1 + \|z_i - c_j\|_2^2/\alpha\right)^{-(\alpha+1)/2}}{\sum_{k=1}^{K} \left(1 + \|z_i - c_k\|_2^2/\alpha\right)^{-(\alpha+1)/2}}.$$

Sharpen these assignments into targets t_{ij}, correcting for the soft cluster sizes n_j:

$$n_j = \sum_{i=1}^{N} r_{ij}, \qquad t_{ij} = \frac{r_{ij}^2/n_j}{\sum_{k=1}^{K} r_{ik}^2/n_k}.$$

Minimise the average KL divergence between categorical assignments:

$$J_{\mathrm{DEC}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} t_{ij} \log \frac{t_{ij}}{r_{ij}}.$$

Hold t_{ij} fixed during each optimisation phase, then recompute the targets periodically. The paper uses $\alpha = 1$.

Xie et al. [XGF16], Eqs. (1)–(3), with renamed symbols and an average loss.

DEC: Updating Codes and Cluster Centres

With the targets held fixed, define

$$A_{ij} = \left(1 + \|z_i - c_j\|_2^2/\alpha\right)^{-1} (t_{ij} - r_{ij}).$$

For the average objective on the previous slide,

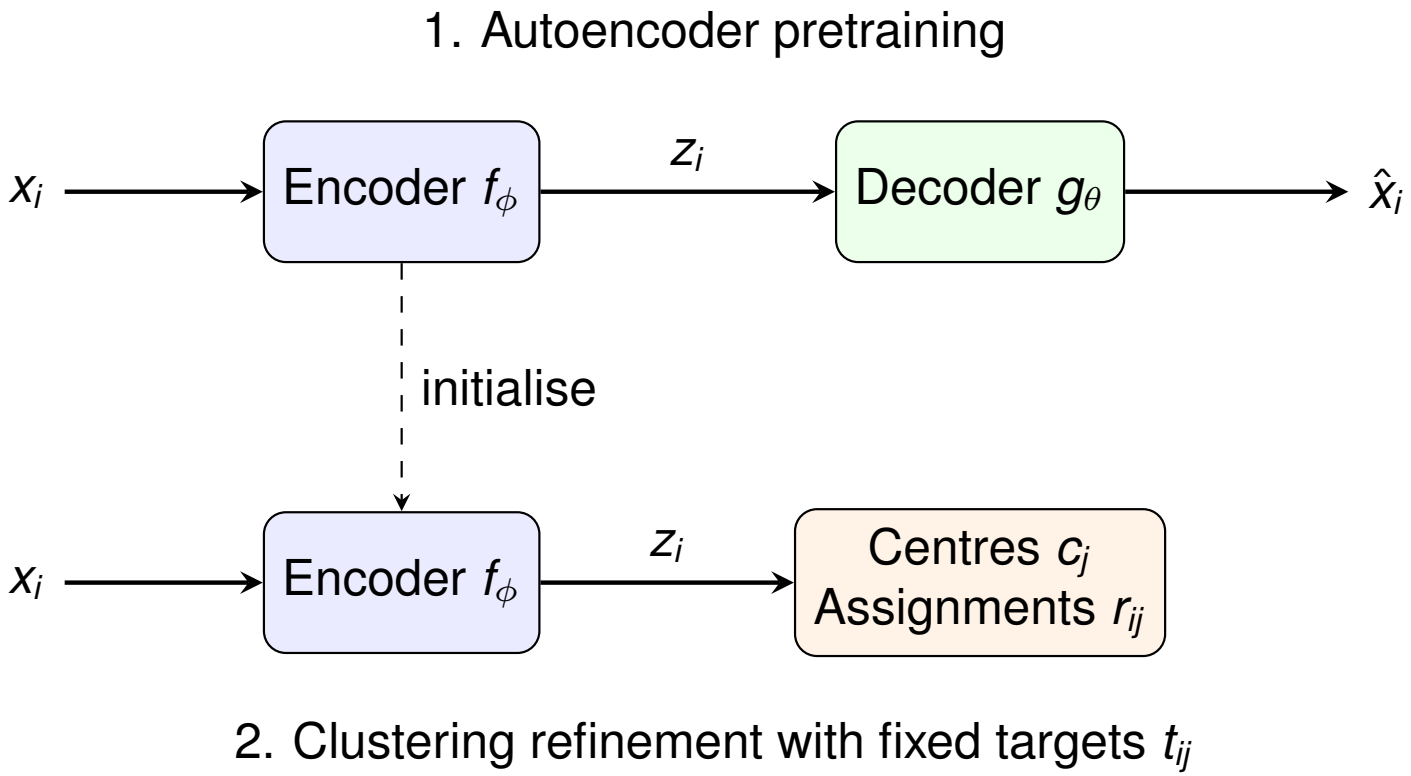
$$\frac{\partial J_{\mathrm{DEC}}}{\partial z_i} = \frac{\alpha + 1}{N\alpha} \sum_{j=1}^{K} A_{ij}(z_i - c_j),$$

$$\frac{\partial J_{\mathrm{DEC}}}{\partial c_j} = -\frac{\alpha + 1}{N\alpha} \sum_{i=1}^{N} A_{ij}(z_i - c_j).$$

Backpropagate through $z_i = f_\phi(x_i)$ to update ϕ. Stop when the fraction of changed hard assignments falls below a chosen tolerance.

Xie et al. [XGF16], Eqs. (4)–(5), rescaled by $1/N$.

DEC: From Reconstruction to Clustering



Redrawn from the procedure in Xie et al. [XGF16].

DEC: Reported Experimental Results

Dataset	# Points	# classes	Dimension	% of largest class
MNIST (LeCun et al., 1998)	70000	10	784	11%
STL-10 (Coates et al., 2011)	13000	10	1428	10%
REUTERS-10K	10000	4	2000	43%
REUTERS (Lewis et al., 2004)	685071	4	2000	43%

Method	MNIST	STL-HOG	REUTERS-10k	REUTERS
k-means	53.49%	28.39%	52.42%	53.29%
LDMGI	84.09%	33.08%	43.84%	N/A
SEC	80.37%	30.75%	60.08%	N/A
DEC w/o backprop	79.82%	34.06%	70.05%	69.62%
DEC (ours)	**84.30%**	**35.90%**	**72.17%**	**75.63%**

Clustering accuracy uses the best one-to-one matching of clusters to reference labels. These are historical benchmark results; “ours” refers to Xie et al.

Xie et al. [XGF16], Tables 1–2. STL uses HOG features; Reuters uses text features. Dataset references: [L98, C11, L04]. Best accuracy across tested settings; not a run average.

DEC: Examples of Learned Clusters



(a) MNIST



(b) STL-10

Each row shows ten highly ranked observations from one cluster. Clusters need not correspond exactly to semantic classes.

Xie et al. [XGF16], Figure 3: MNIST (left), STL-10 (right).

What Does Reconstruction-Based Learning Give Us?

- A way to train representations without externally supplied class labels.
- Features for downstream tasks, whose usefulness depends on the data, architecture and training objective.
- A reconstruction mechanism, but not automatically a model of the full data distribution.

A limitation to carry into the next section

An ordinary AE does not specify a prior distribution over its codes. Low reconstruction error alone does not tell us how to sample new observations.

Next: variational autoencoders

Introduce a latent-variable probability model and train it with approximate inference.

Background: Goodfellow et al. [GBC16].

Outline

1 Autoencoders (AEs)

2 Variational Autoencoders (VAEs)

3 Generative Adversarial Networks (GANs)

From Reconstruction to Generation

- An autoencoder learns to reconstruct an observed input:

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad \min_{\phi,\theta} \sum_i \ell\big(x_i, g_\theta(f_\phi(x_i))\big).$$

- But how should we choose a latent code to generate a *new* sample?
- A standard autoencoder does not specify a distribution from which to sample codes.
- Codes sampled arbitrarily may lie far from the codes seen during training.

The VAE approach

Specify a probabilistic generative model and learn an encoder that helps us train it.

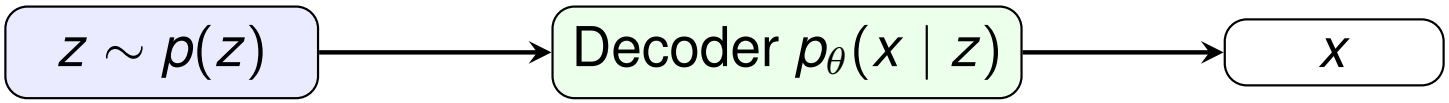
Background: Kingma and Welling [KW14, KW19].

A Generative Model with Latent Variables

To generate an image x, first choose a latent code z, then decode it:

1. Draw a latent variable from a simple **prior**: $z \sim p(z) = \mathcal{N}(0, I_m)$.
2. Draw an observation from a learned **conditional distribution**: $x \sim p_\theta(x \mid z)$.

$$p_\theta(x, z) = p(z)p_\theta(x \mid z), \qquad p_\theta(x) = \int p_\theta(x \mid z)p(z)\, dz.$$



The Gaussian prior $\mathcal{N}(0, I_m)$ has m independent standard normal coordinates. The decoder outputs distribution parameters, such as pixel means or probabilities.

Why Is Maximum Likelihood Difficult?

Given observations $x_1, \ldots, x_N$, we would like to solve

$$\max_\theta \sum_{i=1}^{N} \log p_\theta(x_i) = \max_\theta \sum_{i=1}^{N} \log \int p_\theta(x_i \mid z) p(z)\, dz.$$

- With a nonlinear decoder, this integral usually has no tractable closed form.
- Many latent codes may explain the same observation.
- The exact **posterior** also involves the difficult integral:

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z) p(z)}{p_\theta(x)}.$$

Idea

Learn a tractable approximation $q_\phi(z \mid x)$ to the posterior, and optimise a lower bound on the log-likelihood.

The Encoder Predicts a Distribution

Instead of returning one code, the encoder (with learned parameters ϕ) returns a Gaussian distribution over codes:

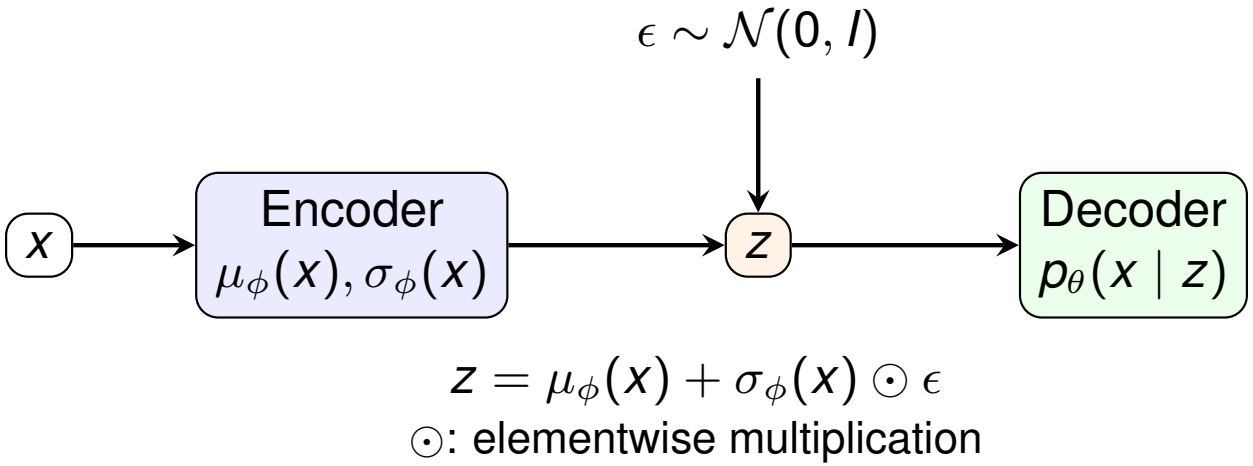
$$q_\phi(z \mid x) = \mathcal{N}\left(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x))\right).$$

- For each input, the encoder predicts a mean and a log-variance vector:

$$\underbrace{\mu_\phi(x)}_{\text{mean}}, \qquad \underbrace{s_\phi(x) = \log \sigma^2_\phi(x)}_{\text{log-variance}}.$$

- Recover positive standard deviations using $\sigma_\phi(x) = \exp(s_\phi(x)/2)$ elementwise.
- Each input is represented by a **distribution over codes**, rather than a single code.

VAE Architecture: Training and Generation



- **Training:** encode an observation, sample a code and evaluate its reconstruction likelihood.
- **Generation:** draw $z \sim p(z)$ and use the decoder; no input observation or encoder is needed.

Two questions

What loss should we optimise? How can gradients pass through sampling?

A Reminder: Kullback–Leibler Divergence

How different are two distributions? For densities q and p,

$$\mathrm{KL}(q\|p) = \mathbb{E}_{z\sim q}\left[\log \frac{q(z)}{p(z)}\right].$$

- $\mathrm{KL}(q\|p) \geq 0$, with equality when $q = p$ almost everywhere.
- Generally $\mathrm{KL}(q\|p) \neq \mathrm{KL}(p\|q)$: it is not a distance metric.
- $\mathbb{E}_{z\sim q}$ means averaging over draws from q; the direction matters.

Two different KL terms will appear

Posterior approximation: $\mathrm{KL}(q_\phi(z \mid x)\|p_\theta(z \mid x))$.

Prior regularisation: $\mathrm{KL}(q_\phi(z \mid x)\|p(z))$.

Deriving the Evidence Lower Bound (ELBO)

For a fixed observation x, insert the encoder density into the integral:

$$\begin{aligned}\log p_\theta(x) &= \log \int q_\phi(z \mid x) \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\, dz \\ &= \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)} \right] \\ &\geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)} \right] =: \mathcal{L}_{\theta,\phi}(x).\end{aligned}$$

- The inequality follows from **Jensen's inequality**: $\log$ is concave.
- $\mathcal{L}_{\theta,\phi}(x)$ is the **evidence lower bound**.
- We maximise this bound over both θ and ϕ.

Kingma and Welling [KW14, KW19].

The ELBO Has Two Terms

Using $p_\theta(x, z) = p_\theta(x \mid z)p(z)$ gives

$$\boxed{\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \| p(z)) .}$$

Reconstruction term

Encourages codes drawn from the encoder to give high likelihood to the observed input under the decoder.

Prior regularisation term

Penalises departures of the encoder distribution from the prior used for generation.

Training balances these two requirements. Perfect reconstruction alone is not the objective.

How Tight Is the Bound?

The exact identity is

$$\log p_\theta(x) = \mathcal{L}_{\theta,\phi}(x) + \mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x)) \,.$$

- The gap is the KL divergence to the **true posterior**.
- The bound is tight if the encoder equals that posterior.
- A restricted encoder family may not represent the true posterior exactly.
- We can optimise the ELBO without evaluating $p_\theta(z \mid x)$.

Keep the two KL terms separate

The *gap* uses the true posterior; the tractable *training objective* uses the prior.

Choosing the Decoder: Binary Observations

For $x \in \{0,1\}^d$, let the decoder output probabilities $\pi_\theta(z)$:

$$p_\theta(x \mid z) = \prod_{k=1}^{d} \pi_{\theta,k}(z)^{x_k} \left(1 - \pi_{\theta,k}(z)\right)^{1-x_k}.$$

Its negative log-likelihood is binary cross-entropy:

$$-\log p_\theta(x \mid z) = -\sum_{k=1}^{d} \left[x_k \log \pi_{\theta,k}(z) + (1 - x_k) \log(1 - \pi_{\theta,k}(z))\right].$$

- Example: binarised handwritten digit images.
- Pixel independence is assumed *conditional on z*; pixels need not be independent marginally.
- A Bernoulli likelihood models binary data. Continuous intensities require a suitable observation model.

Choosing the Decoder: Continuous Observations

For $x \in \mathbb{R}^d$, a simple model is

$$p_\theta(x \mid z) = \mathcal{N}\big(x; g_\theta(z), \tau^2 I_d\big),$$

where $\tau^2 > 0$ is a fixed observation variance. Then

$$-\log p_\theta(x \mid z) = \frac{1}{2\tau^2}\|x - g_\theta(z)\|_2^2 + \frac{d}{2}\log(2\pi\tau^2).$$

- Squared reconstruction error follows from a Gaussian likelihood.
- Its scale relative to the KL term depends on τ^2.
- If the variance is learned, the log-variance term cannot be dropped.
- The decoder mean $g_\theta(z)$ differs from a sample $x \sim p_\theta(x \mid z)$.

The Gaussian KL Term Is Available in Closed Form

Write $\mu = \mu_\phi(x)$ and $\sigma = \sigma_\phi(x)$. For $q_\phi(z \mid x) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I_m)$,

$$\mathrm{KL}(q_\phi(z \mid x) \| p(z)) = \frac{1}{2} \sum_{j=1}^{m} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right).$$

- No Monte Carlo approximation is needed for this term.
- It is minimised at $\mu = 0$ and $\sigma_j^2 = 1$ for every j.
- Driving σ_j^2 to zero incurs an increasingly large penalty.

Quick check: one latent dimension

If $\mu = 2$ and $\sigma^2 = 1$, then $\mathrm{KL}(q \| p) = 2$.
What is the KL divergence when $\mu = 0$ and $\sigma^2 = 1$?

Kingma and Welling [KW14], Appendix B.

The Reparameterisation Trick

We need gradients of an expectation whose sampling distribution depends on ϕ. Instead of treating $z \sim q_\phi(z \mid x)$ as a black-box sample, write

$$\epsilon \sim \mathcal{N}(0, I_m), \qquad \boxed{z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.}$$

- All randomness is in ϵ, whose distribution is independent of ϕ.
- For a fixed draw of ϵ, z is differentiable in the encoder outputs:

$$\frac{\partial z_j}{\partial \mu_j} = 1, \qquad \frac{\partial z_j}{\partial \sigma_j} = \epsilon_j.$$

- Reconstruction gradients can now reach the encoder through z.
- The distribution of z is unchanged by this rewrite.

Kingma and Welling [KW14].

The Loss We Actually Minimise

For a minibatch of B observations, use one latent draw per observation to estimate the negative ELBO:

$$\widehat{J}(\theta, \phi) = \frac{1}{B} \sum_{i=1}^{B} \left[-\log p_\theta(x_i \mid z_i) + \mathrm{KL}(q_\phi(z \mid x_i) \| p(z)) \right],$$

$$z_i = \mu_\phi(x_i) + \exp(s_\phi(x_i)/2) \odot \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I_m).$$

The KL term uses the closed-form Gaussian expression.

- This is an unbiased Monte Carlo estimate of the minibatch negative ELBO.
- Update encoder and decoder together using backpropagation.

A VAE Training Step: Binarised Images

Assume x has shape $[B, d]$ and encoder outputs have shape $[B, m]$.

```
mu, logvar = encoder(x)
std = (0.5 * logvar).exp()
eps = torch.randn_like(std)
z = mu + std * eps
logits = decoder(z)

recon = F.binary_cross_entropy_with_logits(
    logits, x, reduction="none").sum(dim=1)
kl = 0.5 * (mu.square() + logvar.exp()
            - 1.0 - logvar).sum(dim=1)
loss = (recon + kl).mean()

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

PyTorch-style code: `F` denotes `torch.nn.functional`.

The optimiser includes both networks. The decoder returns logits, without a final sigmoid.

Reconstruction, Generation and Interpolation

Reconstruct a given observation

Encode x, draw $z \sim q_\phi(z \mid x)$, then inspect the decoder mean.
Using $z = \mu_\phi(x)$ gives a deterministic reconstruction convention.

Generate a new observation

Draw $z \sim \mathcal{N}(0, I_m)$, then draw $x \sim p_\theta(x \mid z)$.
We often display the decoder's predicted mean image instead of sampling from its output distribution.

Explore the latent space

For two encoded means z_a, z_b, decode

$$z(t) = (1 - t)z_a + tz_b, \qquad t \in [0, 1].$$

This explores the learned representation; plausible intermediate samples are not guaranteed.

Why Does the Prior Matter?

- Training draws latent codes from $q_\phi(z \mid x)$; generation draws them from $p(z)$.
- The KL term encourages these distributions to be compatible.
- Across a dataset, the **aggregate encoder distribution** is

$$q_\phi(z) = \frac{1}{N} \sum_{i=1}^{N} q_\phi(z \mid x_i).$$

- It is generally a mixture of Gaussians.
- The per-observation KL penalty encourages aggregate agreement with the prior, but also limits information about x stored in z.

Note

The objective does not guarantee a perfectly filled latent space, semantic disentanglement, or realistic samples for every code.

Limitations and the Reconstruction–Regularisation Trade-off

- **Posterior collapse:** the decoder may ignore z, with $q_\phi(z \mid x) \approx p(z)$ and a small KL term.
- **Restricted distributions:** a diagonal Gaussian encoder may poorly approximate a complicated posterior.
- **Blurry reconstructions:** simple pixelwise likelihoods and limited latent information can favour averages of plausible outputs.

A common variation changes the strength of regularisation:

$$J_\beta(x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + \beta \,\mathrm{KL}(q_\phi(z \mid x) \| p(z)).$$

- $\beta = 1$: the standard negative ELBO.
- Larger β: stronger prior regularisation, often at a reconstruction cost; disentanglement is not guaranteed.
- Monitor reconstruction and KL separately during training.

Kingma and Welling [KW19]; Higgins et al. [BETA17].

VAE Takeaways and Transition to GANs

1. A VAE learns a **latent-variable generative model** $p(z)p_\theta(x \mid z)$.
2. The encoder $q_\phi(z \mid x)$ performs **approximate posterior inference**.
3. The ELBO balances **reconstruction likelihood** and **KL regularisation**.
4. Reparameterisation enables **gradient-based training** through latent sampling.

Check your understanding

Why is the encoder unnecessary for generation?
What would minimising only the KL term encourage?

Next: a different training signal

Can we train a generator using a learned discriminator to assess its samples?

Outline

1 Autoencoders (AEs)

2 Variational Autoencoders (VAEs)

3 Generative Adversarial Networks (GANs)

GANs: Learning to Generate through Competition

- Goal: generate observations whose distribution resembles the training-data distribution p_{data}.
- A **generator** G_θ, with learned parameters θ, maps random codes to synthetic observations:

$$z \sim p(z), \qquad x_{\text{fake}} = G_\theta(z).$$

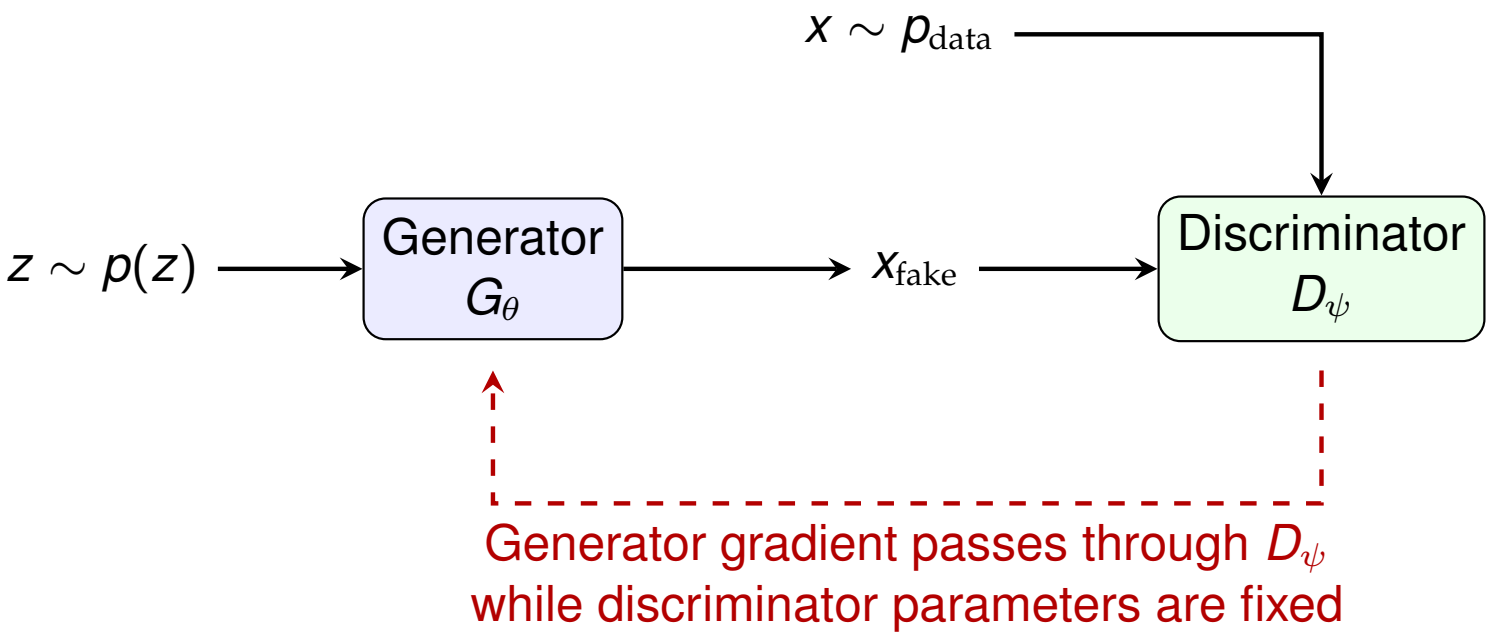
- A **discriminator** D_ψ, with its own learned parameters ψ, learns to distinguish real observations from generated ones.
- The generator learns from the discriminator's feedback.

What if the loss were learned too?

Instead of specifying a reconstruction likelihood as in a VAE, can another network learn to tell us what makes an image look real?

Goodfellow et al. [GAN14].

GAN Architecture and the Gradient Path



- G_θ receives sampled noise; it does not generate its own noise input.
- $D_\psi(x) \in (0, 1)$ estimates the probability of the "real" class in the real/fake classification task.

Architecture redrawn for this lecture; GAN framework: Goodfellow et al. [GAN14].

The Original GAN Minimax Objective

Define the value function

$$V(\psi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D_\psi(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D_\psi(G_\theta(z)))].$$

The original game is

$$\min_\theta \max_\psi V(\psi, \theta).$$

- **Discriminator:** maximise V; assign high scores to real observations and low scores to generated observations.
- **Generator:** minimise V; increase the discriminator's scores on generated observations.
- The real-data term is constant with respect to θ.

Goodfellow et al. [GAN14].

Generator Loss: Minimax versus Non-Saturating

The original minimax generator loss is

$$J_G^{\mathrm{MM}} = \mathbb{E}_{z\sim p(z)}[\log(1 - D_\psi(G_\theta(z)))].$$

If the discriminator confidently rejects generated observations, its sigmoid output can yield weak generator gradients.

Common practical alternative

Minimise the **non-saturating** generator loss:

$$J_G^{\mathrm{NS}} = -\mathbb{E}_{z\sim p(z)}[\log D_\psi(G_\theta(z))].$$

- It supplies a stronger gradient with respect to the discriminator logit when $D_\psi(G_\theta(z))$ is near zero.
- It changes the generator's training objective, while retaining the desired distribution-matching equilibrium.
- A large loss value does not itself guarantee a useful parameter gradient.

Goodfellow et al. [GAN14], Section 3.

Alternating Discriminator and Generator Updates

1. Discriminator update: minimise J_D

Sample real observations x_i and independent latent codes z_i:

$$\widehat{J}_D = -\frac{1}{B}\sum_{i=1}^{B}\left[\log D_\psi(x_i) + \log(1 - D_\psi(G_\theta(z_i)))\right].$$

Update ψ only; treat generated observations as fixed inputs: update only the discriminator, without backpropagating into the generator.

2. Generator update: minimise J_G^{NS}

Sample latent codes and compute

$$\widehat{J}_G^{\mathrm{NS}} = -\frac{1}{B}\sum_{i=1}^{B}\log D_\psi(G_\theta(z_i)).$$

Update θ only. Keep ψ fixed, but retain differentiation through D_ψ to its input.

Use one or more discriminator updates per generator update; the choice affects training.

Goodfellow et al. [GAN14], Algorithm 1 and non-saturating alternative.

What Would an Ideal GAN Learn?

Let p_G denote the distribution induced by $G_\theta(z)$ with $z \sim p(z)$. For fixed G_θ and an optimal discriminator D^* capable of learning any function mapping to $[0, 1]$,

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

- Interpret the ratio using densities with respect to a common measure, where the denominator is positive.
- The ideal distribution-matching solution is $p_G = p_{\text{data}}$; then $D^*(x) = 1/2$ on their support.
- These idealised statements do not guarantee that finite neural networks trained by alternating gradient updates will converge.

A discriminator score of $1/2$ is not a certificate

An undertrained discriminator can also output $1/2$ when the generated distribution is poor.

Goodfellow et al. [GAN14], Proposition 1 and Theorem 1.

GAN Challenges: Instability and Mode Collapse

- **Coupled optimisation:** updating one network changes the other network's objective; training can oscillate or fail to converge.
- **Gradient quality:** the minimax generator loss can saturate; useful gradients also depend on the discriminator's behaviour away from real observations.
- **Mode collapse:** the generator may cover only a few modes of the data distribution, producing little diversity.
- **Evaluation:** realistic individual samples do not establish coverage of the data distribution or absence of memorisation.

Practical consequence

Inspect both sample quality and diversity. Neither training loss alone nor a few selected images provides a complete evaluation.

Goodfellow et al. [GAN14]; Arjovsky et al. [WGAN17].

Conditional GANs: Controlling What Is Generated

Let y be a condition, such as a class label, image or text representation:

$$x_{\text{fake}} = G_\theta(z, y), \qquad D_\psi(x, y) \in (0, 1).$$

The conditional minimax objective is

$$\min_\theta \max_\psi \mathbb{E}_{(x,y)\sim p_{\text{data}}}[\log D_\psi(x, y)] + \mathbb{E}_{y\sim p_{\text{data}}(y),\, z\sim p(z)}[\log(1 - D_\psi(G_\theta(z, y), y))].$$

- Both networks receive the condition, including for generated observations.
- Conditioning encourages the generator to match the data associated with y.
- Labels or paired conditions supply additional information when available.
- Conditioning does not guarantee diversity within a condition or prevent mode collapse.

Mirza and Osindero [CGAN14].

Diffusion Models Are a Separate Generative Approach

- A forward process gradually adds noise to observations.
- A learned reverse process generates observations through successive denoising steps.
- Standard denoising diffusion models do not require a real/fake discriminator or a GAN minimax objective.
- Their quality, diversity and computational cost depend on the model, data and sampling procedure.

GAN takeaways

GANs learn from an adversarial training signal. Conditional models add control, without universal guarantees.

Diffusion: Ho, Jain and Abbeel [DDPM20]. GAN foundations: Goodfellow et al. [GAN14].

Comparing Autoencoders, VAEs and GANs

Model	Training signal	Generation
AE	Reconstruction loss, possibly regularised.	Needs an additional way to sample latent codes.
VAE	Reconstruction likelihood and KL regularisation (ELBO).	Draw $z \sim p(z)$, then $x \sim p_\theta(x \mid z)$.
GAN	Adversarial feedback from a discriminator or critic.	Draw $z \sim p(z)$ and set $x = G_\theta(z)$.

- Representation quality, reconstruction quality and generation quality are distinct criteria.
- Evaluate the property required by the application, rather than relying on training loss alone.

Thank you for your attention!

johanneslutzeyer.com

References: Autoencoders

[GBC16] I. Goodfellow, Y. Bengio and A. Courville (2016). *Deep Learning*. MIT Press, Chapter 14.
https://www.deeplearningbook.org/contents/autoencoders.html

[HS06] G. E. Hinton and R. R. Salakhutdinov (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786), 504–507.
https://doi.org/10.1126/science.1127647

[Ng11] A. Ng (2011). *Sparse Autoencoder*. CS294A lecture notes, Stanford University.
https://web.stanford.edu/class/cs294a/sparseAutoencoder_2011new.pdf

[V10] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol (2010). Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion. *JMLR*, 11, 3371–3408.
https://www.jmlr.org/papers/v11/vincent10a.html

References: Clustering and Datasets

[XGF16] J. Xie, R. Girshick and A. Farhadi (2016). Unsupervised Deep Embedding for Clustering Analysis. *ICML*, PMLR 48, 478–487.
https://proceedings.mlr.press/v48/xieb16.html

[L98] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
https://doi.org/10.1109/5.726791

[C11] A. Coates, H. Lee and A. Y. Ng (2011). An Analysis of Single-Layer Networks in Unsupervised Feature Learning. *AISTATS*, PMLR 15, 215–223.
https://proceedings.mlr.press/v15/coates11a.html

[L04] D. D. Lewis, Y. Yang, T. G. Rose and F. Li (2004). RCV1: A New Benchmark Collection for Text Categorization Research. *JMLR*, 5, 361–397.
https://www.jmlr.org/papers/v5/lewis04a.html

References: Variational Autoencoders

[KW14] D. P. Kingma and M. Welling (2014). Auto-Encoding Variational Bayes. *ICLR*.
https://arxiv.org/abs/1312.6114

[KW19] D. P. Kingma and M. Welling (2019). An Introduction to Variational Autoencoders. *Foundations and Trends in Machine Learning*, 12(4), 307–392.
https://doi.org/10.1561/2200000056

[BETA17] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed and A. Lerchner (2017). β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR*.
https://openreview.net/forum?id=Sy2fzU9gl

References: GAN Objectives

[GAN14] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems*, 27.
NeurIPS proceedings

[CGAN14] M. Mirza and S. Osindero (2014). Conditional Generative Adversarial Nets. *arXiv:1411.1784*.
https://arxiv.org/abs/1411.1784

[WGAN17] M. Arjovsky, S. Chintala and L. Bottou (2017). Wasserstein Generative Adversarial Networks. *ICML*, PMLR 70, 214–223.
https://proceedings.mlr.press/v70/arjovsky17a.html

[GP17] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin and A. Courville (2017). Improved Training of Wasserstein GANs. *Advances in Neural Information Processing Systems*, 30.
https://arxiv.org/abs/1704.00028

References: Other Generative Models

[DDPM20] J. Ho, A. Jain and P. Abbeel (2020). Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems*, 33, 6840–6851. https://arxiv.org/abs/2006.11239

[R21] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen and I. Sutskever (2021). Zero-Shot Text-to-Image Generation. *ICML*, PMLR 139, 8821–8831. https://proceedings.mlr.press/v139/ramesh21a.html

The original DALL-E [R21] combines a discrete VAE with an autoregressive Transformer; it is not a GAN.